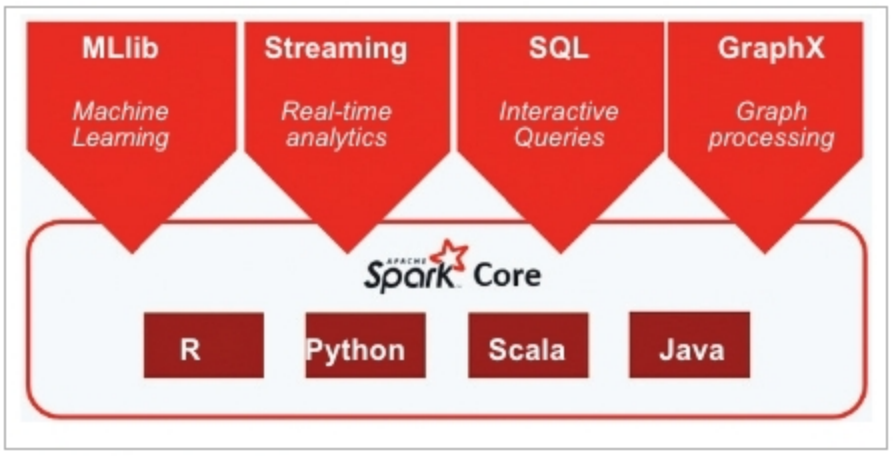

Figure 5: Spark extensions

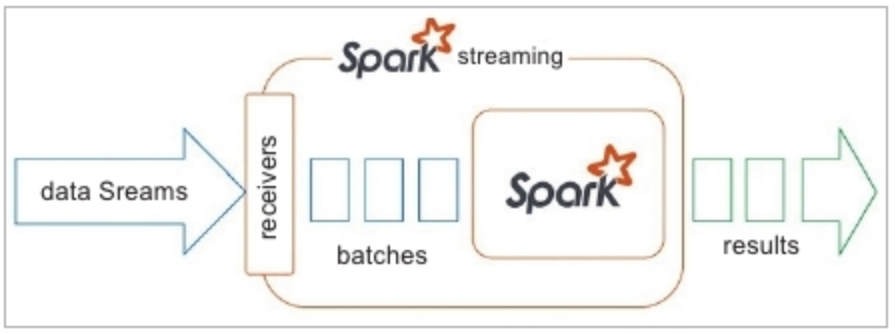

Figure 6: Spark streaming

YARN cluster deployment mode: In cluster mode, the Spark driver runs in the ApplicationMaster on a cluster host. A single process in a YARN container is responsible for both driving the application and requesting resources from YARN. The client that launches the application does not need to run for the lifetime of the application.

Cluster mode is not well suited to using Spark interactively. Spark applications that require user input, such as *spark-shell* and *pyspark*, require the Spark driver to run inside the client process that initiates the Spark application.

YARN client deployment mode: In client mode, the Spark driver runs on the host where the job is submitted. The ApplicationMaster is responsible only for requesting executor containers from YARN. After the containers start, the client communicates with the containers to schedule work.

Why Java and Scala are preferable for Spark application development

Accessing Spark with Java and Scala offers many advantages. They are:

1. Platform independence by running inside the JVM, self-contained packaging of code and its dependencies into JAR files, and higher performance because Spark itself runs in the JVM. You lose these advantages when using the Spark Python API.
2. Managing dependencies and making them available for Python jobs on a cluster can be difficult.
3. To determine which dependencies are required on the cluster, we must understand that Spark code applications run in the Spark executor processes distributed throughout the cluster.
4. If the Python transformations we define use any third party libraries, such as NumPy or nltk, Spark executors require access to those libraries when they run on remote executors.

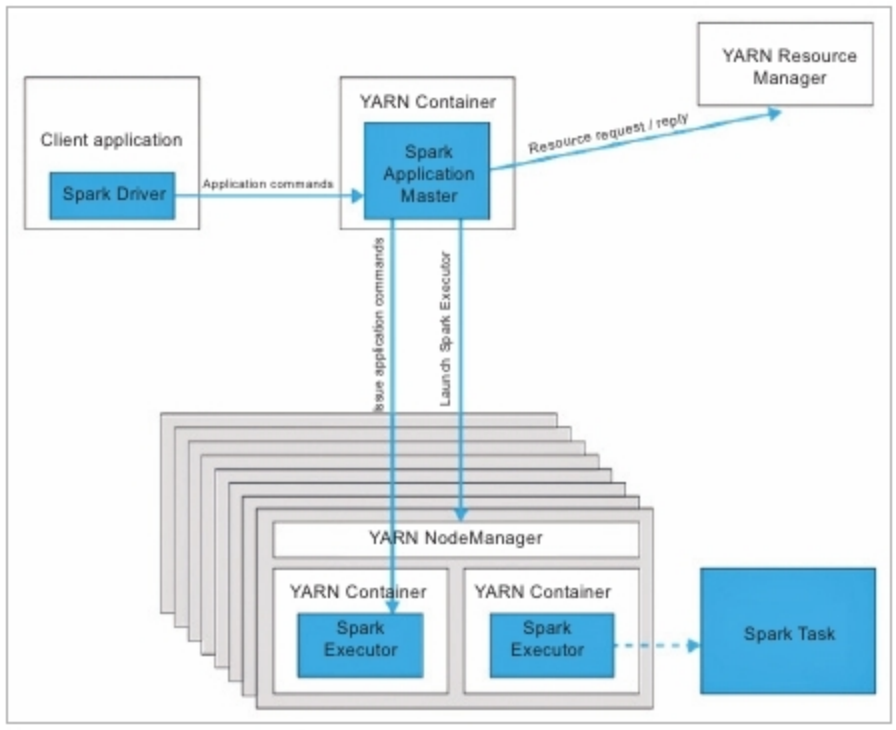

Figure 7: Spark in YARN client mode

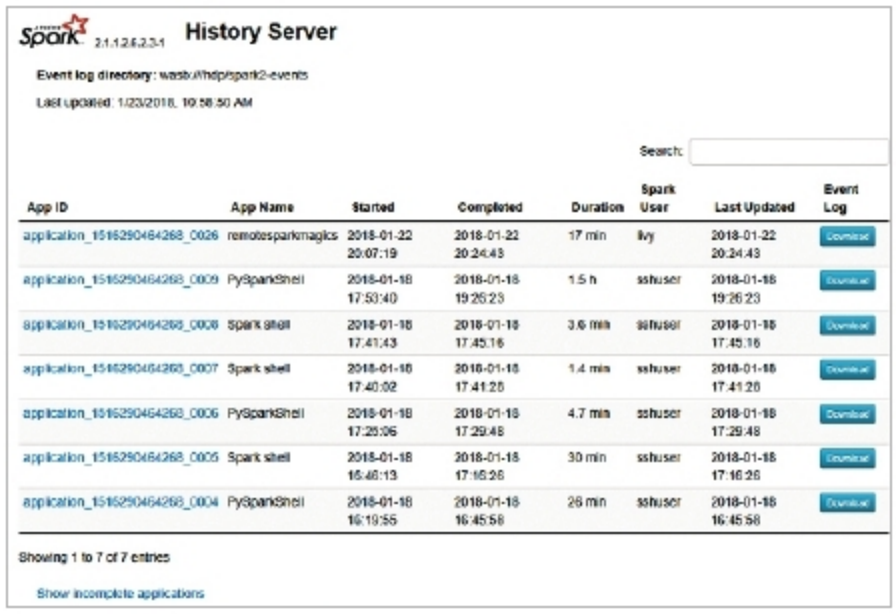

Figure 8: Spark history server Web UI

It is better to implement a shared Python lib path in worker modes using nfs to access the third party libraries.

Spark extensions

Apache Spark has 4 extensions. They are given below.

- Spark Streaming - To process a streaming data
- Spark SQL - To process a structured data
- Spark Mlib - To perform machine learning
- Spark GraphX - To analyse graph data

Please refer to Figure 6.

Spark Streaming

Spark Streaming is an extension of the core Spark API that enables scalable, high-throughput, fault-tolerant stream processing of live data streams. Data can be ingested from many sources like Kafka, Flume, Kinesis or TCP sockets, and can be processed using complex algorithms expressed with high-level functions like MapReduce, Join and Window. Finally, processed data can be pushed out to file systems, databases and live dashboards. We can apply Spark's machine learning and graph processing algorithms on data streams.